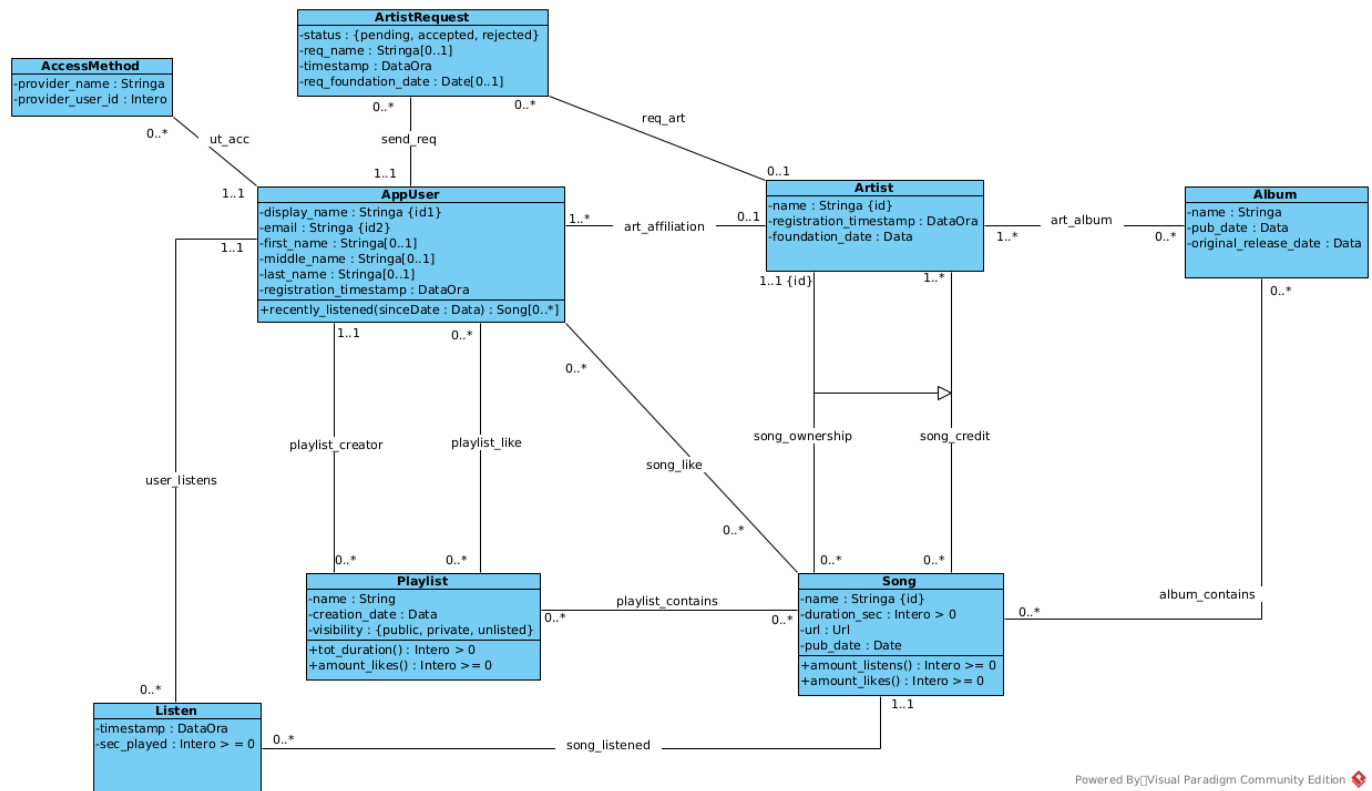


Indice

- **Analisi Concettuale dei Requisiti**
 - Diagramma delle Classi (Class Diagram)
 - Specifica dei tipi di dato
 - Vincoli Esterni
 - [V.Artist.registrato_dopo_fondazione]
 - [V.art_album.pubblicazione_dopo_fondazione]
 - [V.song_credit.pubblicazione_dopo_fondazione]
 - [V.Album.rilascio_originale_prima_di_pubblicazione]
 - [V.art_album.artista_fondato_prima_rilascio_album]
 - [V.art_album.artista_registrato_prima_pubblicazione_album]
 - [V.Listen.data_di_ascolto_valida]
 - [V.Listen.non_ascolta_piu_della_durata]
 - [V.ArtistRequest.tipo_richiesta_esclusivo]
 - [V.ArtistRequest.richiesta_dopo_registrazione_ut]
 - [V.ArtistRequest.richiesta_dopo_registrazione_art]
 - Specifica delle Classi
 - Specifica della classe AppUser
 - Specifica della classe Playlist
 - Specifica della classe Song
 - Diagramma degli Use-Case
 - Specifica degli Use-Case
 - Strumenti di Autenticazione e Autorizzazione
 - Strumenti di Utilizzo Playlist e Musiche
 - Strumenti di Interazione
 - Use-Case Strumenti Gestione Catalogo Musicale
 - Use-Case Strumenti Richiesta Artista
 - Use-Case Strumenti di Moderazione
- **Ristrutturazione**
 - Diagramma delle Classi Ristrutturato
 - Specifica dei tipi di dato (Ristrutturata)
 - Vincoli Ristrutturati
 - [V.song_owner.IS_A_song_credit]
 - Specifica delle Classi
 - Specifica della classe Song
 - Specifica della classe Playlist
 - Implementazione dei Vincoli

Analisi Concettuale dei Requisiti

Diagramma delle Classi (Class Diagram)



Powered By [Visual Paradigm Community Edition]

Specifica dei tipi di dato

- Url: Stringa che rappresenta un URL http o https secondo la sintassi RFC 3986

Vincoli Esterni

[V.Artist.registrato_dopo_fondazione] Un Artista può registrarsi solo dopo la sua data di fondazione

Per ogni $a:Artist$ deve essere vero che $a.foundation_date \leq a.registration_timestamp$

[V.art_album.pubblicazione_dopo_fondazione] Un Artista non può pubblicare un Album se non è stato fondato

Per ogni $art:Artist$ e $alb:Album$, tali che $(art, alb):art_album$, deve essere vero che $art.foundation_date < alb.pub_date$

[V.song_credit.pubblicazione_dopo_fondazione] Un Artista non può pubblicare una Canzone se non è stato fondato

Per ogni $a:Artist$ e $s:Song$, tali che $(a, s):song_credit$, deve essere vero che $a.foundation_date \leq s.pub_date$

[V.Album.rilascio_originale_prima_di_pubblicazione] Un Album può essere pubblicato sulla piattaforma solo durante o dopo il suo rilascio

Per ogni $al:Album$, deve essere vero che $al.original_release_date \leq al.pub_date$

[V.art_album.artista_fondato_prima_rilascio_album] Il rilascio ufficiale di un Album deve avvenire dopo la fondazione dei suoi Artisti

Per ogni *al:Album* e *art:Artist*, tali che *(al, art):art_album*, deve essere vero che *art.foundation_date* <= *al.pub_date*

[V.art_album.artista_registrato_prima_pubblicazione_album] La pubblicazione di un Album può avvenire solo da Artisti registrati prima della data di pubblicazione dell'Album stesso

Per ogni *al:Album* e *art:Artist*, tali che *(al, art):art_album*, deve essere vero che *art.registration_timestamp* < *al.pub_date*

[V.Listen.data_di_ascolto_valida] Un Ascolto deve esser fatto dopo la registrazione di un Utente e dopo la pubblicazione di una Canzone

Per ogni *u:AppUser*, *l:Listen* e *s:Song*, tali che *(u, l):user_listens* e che *(s, l):song_listened*, devono essere vere entrambe le condizioni:

- *s.pub_date* <= *l.timestamp*
- *u.registration_timestamp* <= *l.timestamp*

[V.Listen.non_ascolta_piu_della_durata] Un Utente non può Ascoltare per una durata superiore alla durata della Canzone stessa

Per ogni *l:Listen* e *s:Song*, tali *(s, l):song_listened*, deve essere vero che *l.sec_played* <= *s.duration_sec*

[V.ArtistRequest.tipo_richiesta_esclusivo] Una Richiesta deve indicare il nome per un nuovo Artista oppure riferirsi a un Artista già esistente, ma non entrambe o nessuna delle due.

Per ogni *r:ArtistRequest*, deve essere vera esattamente una e una sola (**xor**) delle due seguenti condizioni:

- Esiste un valore per l'attributo *r.req_name* e, contemporaneamente, un valore per l'attributo *r.req_foundation_date*
- Esiste un *a:Artist*, tale che esista il link *(r, a):req_art*

[V.ArtistRequest.richiesta_dopo_registrazione_ut] Un Utente non può Richiedere di diventare un Artista prima della sua registrazione

Per ogni *r:ArtistRequest* e *u:AppUser*, tali che esista il link *(r, u):send_req* deve essere vero che *u.registration_timestamp* < *r.timestamp*

[V.ArtistRequest.richiesta_dopo_registrazione_art] Una Richiesta non può riguardare un Artista che si è registrato dopo la Richiesta stessa

Per ogni *r:ArtistRequest* e *a:Artist*, tali che esista il link *(r, a):req_art* deve essere vero che *a.registration_timestamp* < *r.timestamp*

Specifica delle Classi

Specifica della classe AppUser

recently_listened(sinceDate : Data): Song[0..*]

Pre: Deve essere vero che *sinceDate* < Oggi

Post: L'operazione non modifica i dati. Il risultato è così definito:

- Sia L l'insieme di tutti i $l:Listen$ tali che esiste il link $(this, l):user_listens$ e per i quali sia vero che **sinceDate** < $l.timestamp$
- **result** è l'insieme di tutte le $s:Song$, tali che esiste il link $(l, s):song_listened$, per almeno un oggetto l appartenente all'insieme L

Specifica della classe Playlist

tot_duration(): Intero > 0

Pre: Nessuna

Post: L'operazione non modifica i dati. Il risultato è così definito:

- Sia S l'insieme di tutte le $s:Song$, tali che esiste il link $(this, s):playlist_contains$
- **result** è la somma di tutte le $s.duration_sec$ in S , espresso in ore, minuti e secondi

amount_likes(): Intero > 0

Pre: Nessuna

Post: L'operazione non modifica i dati. Il risultato è così definito:

- Sia $UserLikes$ l'insieme di tutti gli $u:AppUser$, tali che esiste il link $(u, this):playlist_like$
- **result** = $|UserLikes|$

Specifica della classe Song

amount_listens(): Intero > 0

Pre: Nessuna

Post: L'operazione non modifica i dati. Il risultato è così definito:

- Sia L l'insieme di tutti i $l:Listen$, tali che esista il link $(l, this):song_listened$
- **result** = $|L|$

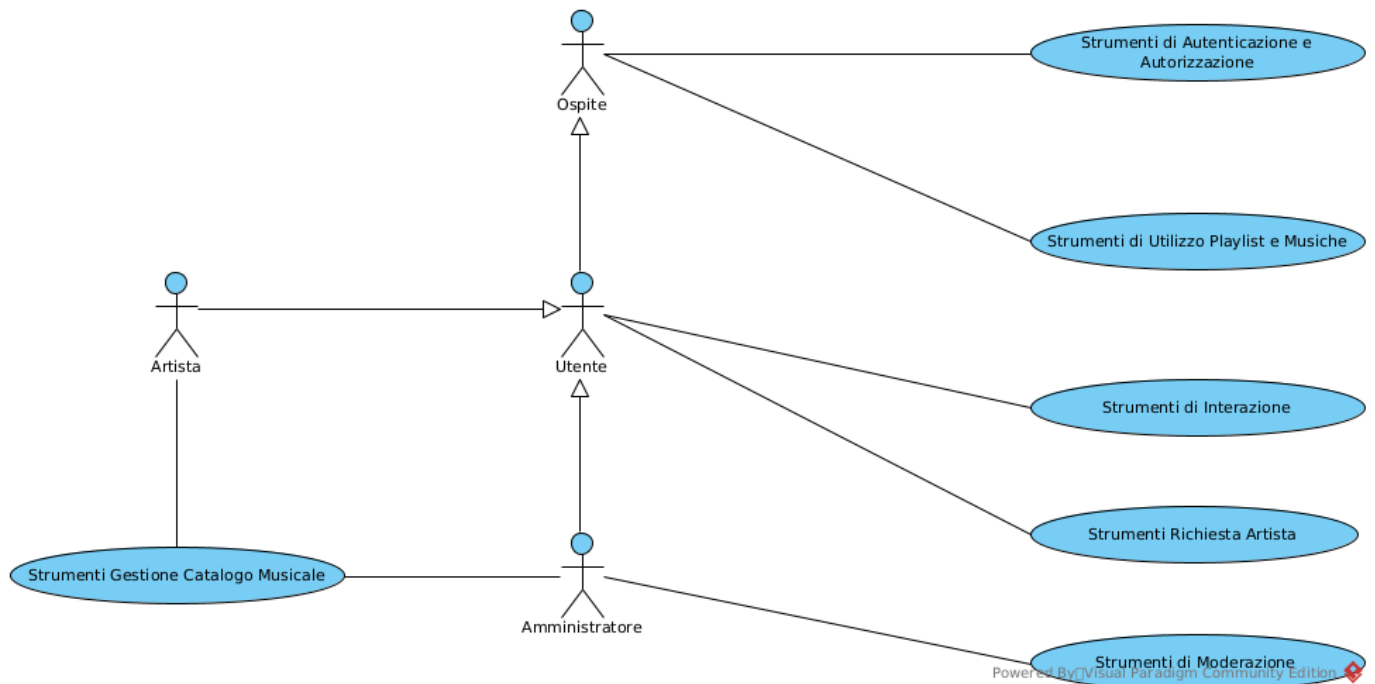
amount_likes(): Intero > 0

Pre: Nessuna

Post: L'operazione non modifica i dati. Il risultato è così definito:

- Sia $UserLikes$ l'insieme di tutti gli $u:AppUser$, tali che esiste il link $(u, this):song_like$
- **result** = $|UserLikes|$

Diagramma degli Use-Case



Specifica degli Use-Case

Strumenti di Autenticazione e Autorizzazione

registra(email: Stringa, disp_name: Stringa, f_name: Stringa[0..1], m_name: Stringa[0..1], l_name: Stringa[0..1]) : AppUser

Pre:

- Non deve esistere alcun *u:AppUser* tale che *u.email* = **email**
- Non deve esistere alcun *u:AppUser* tale che *u.display_name* = **disp_name**
- **email** e **disp_name** non devono essere vuoti

Post:

- Viene creato e restituito un nuovo oggetto *res:AppUser*, con valori *res.email* = **email**, *res.display_name* = **disp_name**, *res.first_name* = **f_name**, *res.middle_name* = **m_name**, *res.last_name* = **l_name** e *res.registration_timestamp* = Oggi in questo momento

Strumenti di Utilizzo Playlist e Musiche

crea_playlist(ut: AppUser, name: Stringa, vis: Stringa) : Playlist

Pre:

- Il valore di **vis** deve appartenere all'insieme {public, private, unlisted}
- **name** non deve essere vuoto

Post:

- Viene creato e restituito un nuovo oggetto *res:Playlist*, con valori *res.name* = **name**, *res.creation_date* = Oggi e *res.visibility* = **vis**
- Viene creato il link *(ut, res):playlist_creator*

aggiungi_branco_a_playlist(ut: AppUser, pl: Playlist, s: Song)

Pre:

- Deve esistere il link *(ut, pl):playlist_creator*
- Non deve esistere il link *(pl, s):playlist_contains*

Post:

- Viene creato il link *(pl, s):playlist_contains*

Strumenti di Interazione

registra_ascolto(ut: AppUser, s: Song, sec: Intero)

Pre:

- Deve essere vero che **sec** >= 0 e **sec** <= s.duration_sec

Post:

- Viene creato un nuovo oggetto *l:Listen*, con valori l.timestamp = Oggi in questo momento e l.sec_played = **sec**
- Viene creato il link *(ut, l):user_listens*
- Viene creato il link *(s, l):song_listened*

like_canzone(ut: AppUser, s: Song)

Pre:

- Non deve esistere il link *(ut, s):song_like*

Post:

- Viene creato il link *(ut, s):song_like*

like_playlist(ut: AppUser, pl: Playlist)

Pre:

- Non deve esistere il link *(ut, pl):playlist_like*

Post:

- Viene creato il link *(ut, pl):playlist_like*

Use-Case Strumenti Gestione Catalogo Musicale

pubblica_canzone(art_owner: Artist, art_credits: Artist[1..*], song_name: Stringa, duration: Intero >= 0, audio_url: Url): Song

Pre:

- **art_owner** deve essere incluso nell'insieme degli artisti **art_credits**
- Per ogni *art:Artist* in **art_credits**, deve essere vero che art.foundation_date <= Oggi
- **song_name** non deve essere vuoto

Post:

- Viene creato e restituito un nuovo oggetto *res:Song*, con valori *res.name* = **song_name**, *res.duration_sec* = **duration**, *res.url* = **audio_url** e *res.pub_date* = Oggi
- Per ogni *art:Artist* in **art_credits**, vengono creati i link *(art, res):song_credit*
 - Tra questi, il link *(art_owner, res)* viene specializzato come istanza anche dell'associazione *song_ownership*

pubblica_album(artists: Artist[1..*], songs: Song[0..*], alb_name: Stringa, orig_rel_date: Data): Album
Pre:

- **alb_name** non deve essere vuoto
- **orig_rel_date** <= Oggi
- Per ogni *art:Artist* in **artists**, deve essere vero che *art.foundation_date* <= **orig_rel_date**

Post:

- Viene creato e restituito un nuovo oggetto *res:Album*, con valori *res.name* = **alb_name**, *res.original_release_date* = **orig_rel_date** e *res.pub_date* = Oggi
- Per ogni *art:Artist* in **artists**, vengono creati i link *(art, res):art_album*
- Per ogni *s:Song* in **songs**, vengono creati i link *(res, s):album_contains*

Use-Case Strumenti Richiesta Artista

invia_richiesta_nuovo_artista(ut: AppUser, req_name: Stringa, req_foundation_date : Data) : ArtistRequest
Pre:

- Non deve esistere alcun *r:ArtistRequest* tale che *(ut, r):send_req* e che (*r.status* = 'pending' oppure *r.status* = 'accepted').
- **req_name** non deve essere vuoto
- Deve essere vero che **req_foundation_date** <= Oggi

Post:

- Viene creato e restituito un nuovo oggetto *res:ArtistRequest*, con valori *res.status* = 'pending', *res.req_name* = **req_name**, *res.req_foundation_date* = **req_foundation_date** e *res.timestamp* = Oggi in questo momento
- Viene creato il link *(ut, res):send_req*

invia_richiesta_artista_esistente(ut: AppUser, art: Artist) : ArtistRequest
Pre:

- Non deve esistere alcun *r:ArtistRequest* tale che *(ut, r):send_req* e che (*r.status* = 'pending' oppure *r.status* = 'accepted').

Post:

- Viene creato e restituito un nuovo oggetto *res:ArtistRequest*, con valori *res.status* = 'pending', *res.req_name* = **req_name** e *res.timestamp* = Oggi in questo momento
- Viene creato il link *(ut, res):send_req*
- Viene creato il link *(res, art):req_art*

Use-Case Strumenti di Moderazione

valuta_richiesta(req: ArtistRequest, accepted: Booleano)

Pre:

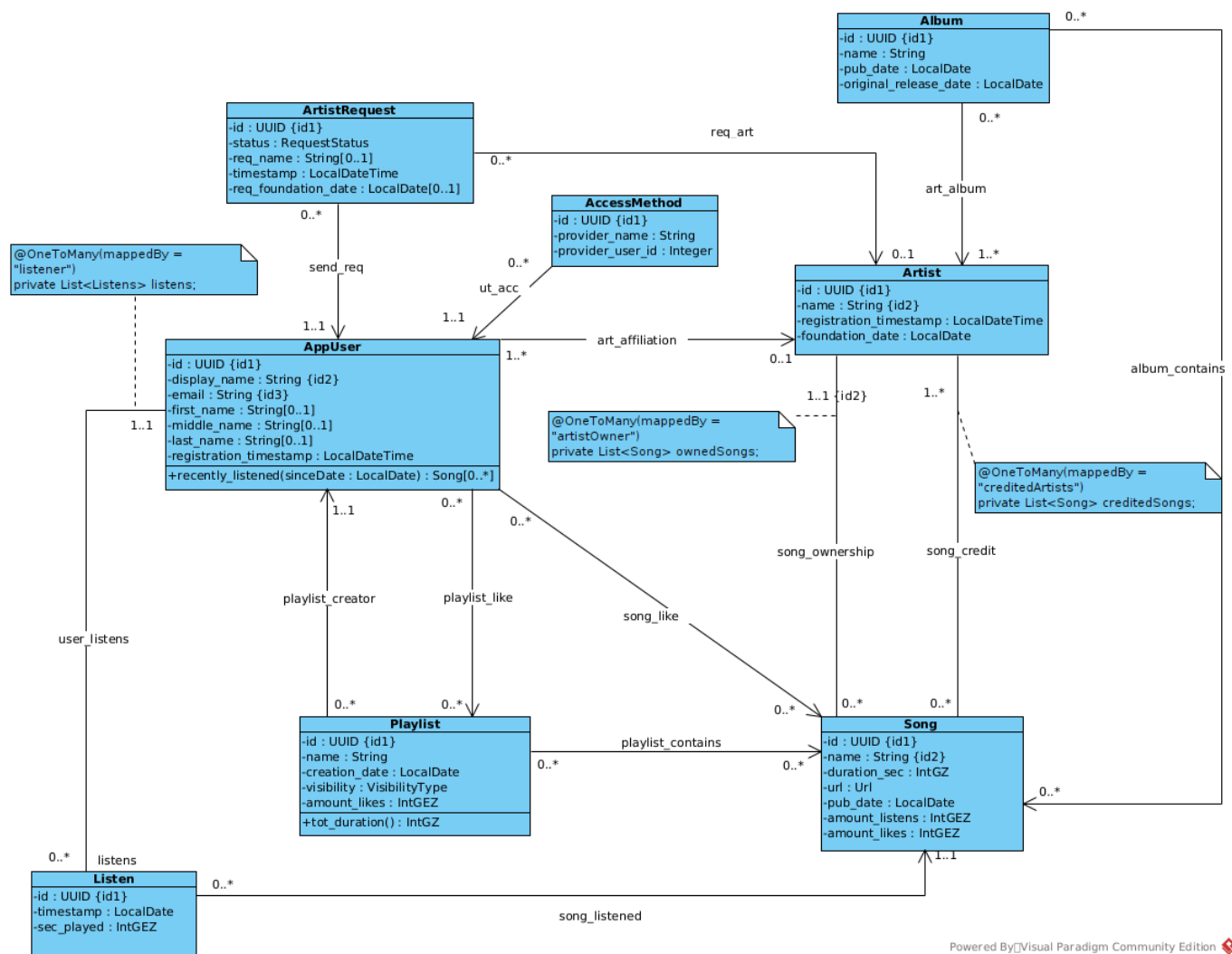
- Deve essere vero che **req.status** = 'pending'

Post:

- Se il valore di **accepted** è falso, allora **req.status** diventa uguale a 'rejected'
- Se il valore di **accepted** è vero, allora **req.status** diventa uguale a 'accepted'. Inoltre:
 - Sia *u:AppUser*, tale che esista il link *(u, req):send_req*
 - Se esiste un *art:Artist*, tale che esista il link *_(req, art):req_art*, viene creato il link *(u, art):art_affiliation*
 - Se non esiste alcun link dell'associazione req_art che coinvolga **req**
 - Viene creato un nuovo oggetto *new_art:Artist*, con valori *new_art.name* = **req.req_name**, *new_art.foundation_date* = **req.req_foundation_date** e *new_art.registration_timestamp* = Oggi in questo momento

Ristrutturazione

Diagramma delle Classi Ristrutturato



Powered By [Visual Paradigm Community Edition]

Specifica dei tipi di dato (Ristrutturata)

- ```
@Positive
@Column(nullable = false)
private Integer nomeAttributo;
```

- ```
@PositiveOrZero
@Column(nullable = false)
private Integer nomeAttributo;
```

- ```
public enum RequestStatus {
 PENDING,
 ACCEPTED,
 REJECTED
}
```

- ```
public enum VisibilityType {
    PUBLIC,
    PRIVATE,
    UNLISTED
}
```

- ```
@Pattern(
 regexp = "https?:\\(www\\\\\\\\\\\\\\\\\\\\)?[-a-zA-Z0-9@:%._\\\\\\\\\\\\\\\\+~#=#]
{1,256}\\\\\\\\\\\\\\\\\\\\[a-zA-Z0-9()]{1,6}\\\\\\\\\\\\\\\\\\\\b([-a-zA-Z0-9()@:%._\\\\\\\\\\\\\\\\+~#?&/=]*)"
)
private String url;
```

*NB: Solo i nuovi vincoli o quelli che sono cambiati rispetto all'analisi verranno riportati qui. Se un vincolo non è riportato, la sua logica non è cambiata*

[V.song\_owner.IS\_A\_song\_credit] Un proprietario del brano deve anche essere un artista accreditato

Per ogni *art:Artist* e *s:Song*, tali che *(art, s):song\_ownership*, deve esistere anche il link *(art, s):song\_credit*

## Specifica delle Classi

*NB: Solo le nuove specifiche o quelle che sono cambiati rispetto all'analisi verranno riportati qui. Se una specifica non è riportata, la sua logica non è cambiata*

### Specifica della classe Song

Nel passaggio all'implementazione relazionale, le operazioni di calcolo aggregato sono state sostituite da attributi a ridondanza controllata per ottimizzare le prestazioni in lettura del database:

**Song.amount\_listens** e **Song.amount\_likes**: Sostituiscono le operazioni correlate dell'analisi concettuale. Questi campi verranno aggiornati dinamicamente dall'applicazione ad ogni nuovo inserimento o rimozione delle entità/relazioni **Listen** e **song\_like**.

### Specifica della classe Playlist

Nel passaggio all'implementazione relazionale, le operazioni di calcolo aggregato sono state sostituite da attributi a ridondanza controllata per ottimizzare le prestazioni in lettura del database:

**Playlist.amount\_likes**: Sostituisce l'operazione correlata dell'analisi concettuale. Questo campo verrà aggiornato dinamicamente dall'applicazione ad ogni nuovo inserimento o rimozione della relazione **playlist\_like**

## Implementazione dei Vincoli

Nel passaggio al design con Spring JPA e database relazionale, l'integrità del modello sarà garantita su più livelli:

- Vincoli di dominio: saranno controllati nell'applicazione usando le annotazioni di Bean Validation (es. @Positive, @Pattern) nelle classi @Entity.
- Vincoli strutturali: saranno controllati dal database tramite Foreign Key e vincoli UNIQUE.
- Vincoli complessi inter-classe e temporali: saranno controllati con regole più complesse, tramite la logica dei metodi dello strato @Service prima di salvare i dati nel database.